

C# • .NET FRAMEWORK

Criando uma Tela de Login

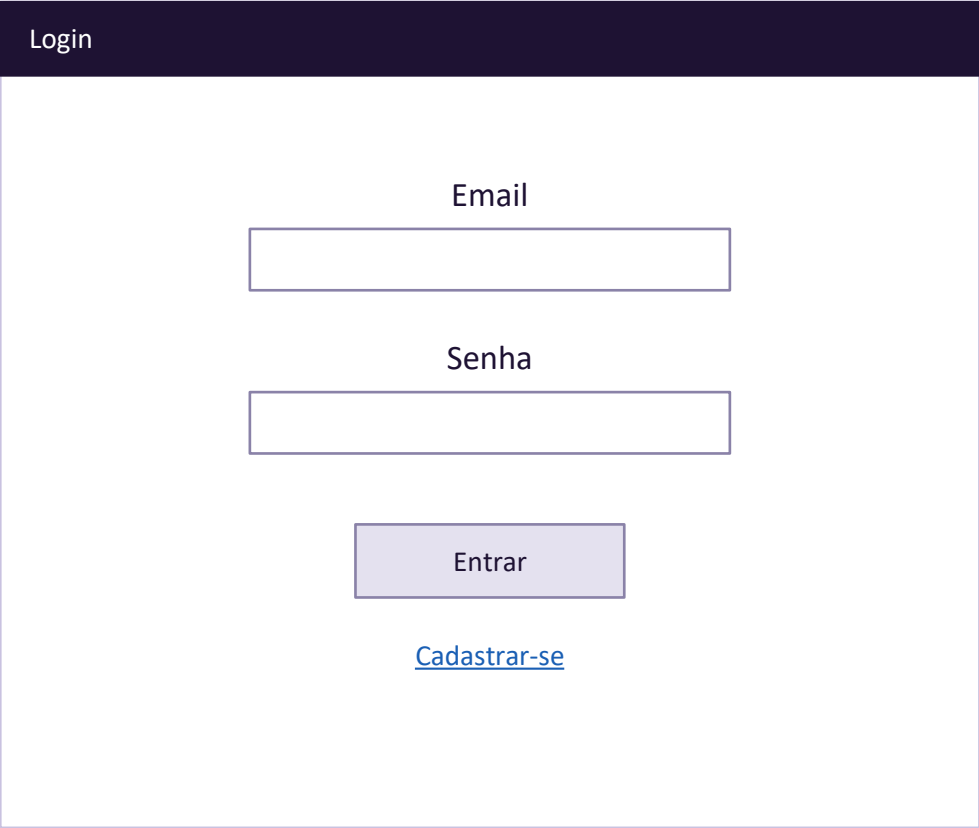
Windows Forms na prática: do primeiro controle
à validação do usuário

Aula passo a passo

O que vamos construir

Uma tela de login completa, feita do zero, com os controles básicos do Windows Forms.

- **Campos de email e senha** com rótulos e senha protegida
- **Botão Entrar** que valida os dados digitados
- **Mensagens ao usuário** avisando erro ou sucesso
- **Link de cadastro** que leva a outra tela
- **Layout que se adapta** sempre centralizado ao redimensionar



The image shows a mockup of a login form. It has a dark blue header bar with the word "Login" in white. Below the header, the form is centered on a white background. It consists of two text input fields, one labeled "Email" and one labeled "Senha" (Password). Below the password field is a light blue button labeled "Entrar" (Enter). At the bottom, there is a blue hyperlink labeled "Cadastrar-se" (Sign up).

Os controles desta aula

Cinco peças resolvem praticamente qualquer formulário. Vamos conhecer uma de cada vez.

Label

Texto fixo na tela.
O usuário lê, mas não edita.

TextBox

Caixa onde o usuário digita a informação.

Button

Botão que dispara uma ação ao ser clicado.

LinkLabel

Texto clicável, com cara de link.

Panel

Caixa que agrupa outros controles.

Todo controle tem propriedades. Propriedade é uma característica que você pode mudar: o texto, a cor, o tamanho, se aparece ou não. Você altera na janela Propriedades do Visual Studio ou por código.

Label — o texto da tela

Serve para identificar campos, exibir títulos e mostrar mensagens. O usuário não consegue editar.

Text

o que aparece escrito

Font

fonte, tamanho e negrito

ForeColor

cor da letra

TextAlign

alinhamento dentro do controle

AutoSize

se o tamanho segue o texto

Dica de sala

Deixe o Label com a mesma largura do TextBox, o mesmo Left e TextAlign = MiddleCenter. Assim ele nunca desalinha do campo.

```
lblEmail.Text = "Email";  
lblEmail.ForeColor = Color.Black;
```

O AutoSize costuma confundir: com ele ligado, o Label ignora a largura que você definiu e encolhe até o tamanho do texto.

TextBox — a entrada de dados

É onde o usuário digita. Tudo que ele escreve fica guardado na propriedade Text.

Text

o conteúdo digitado

PasswordChar

esconde a digitação atrás de um símbolo

MaxLength

limite de caracteres

Multiline

permite várias linhas

ReadOnly

só leitura, não deixa digitar

Campo de senha

No TextBox da senha, defina PasswordChar como • ou marque UseSystemPasswordChar = True. Sem isso, a senha aparece na tela para qualquer um que estiver olhando.

```
// lendo o que foi digitado
string email = txtEmail.Text;

// limpando o campo
txtSenha.Clear();
```

O TextBox devolve sempre texto (string), mesmo quando o usuário digita números.

Button — o botão que executa

O botão não faz nada sozinho. Ele avisa que foi clicado, e você escreve o que deve acontecer.

O evento Click

Dê duplo clique no botão no designer. O Visual Studio cria sozinho o método que roda quando alguém clicar nele.

Propriedade Enabled

Com Enabled = False o botão fica cinza e não responde ao clique. Útil enquanto o formulário ainda não está preenchido.

AcceptButton

É uma propriedade do formulário, não do botão. Apontando para btnEntrar, a tecla Enter já dispara o login.

```
private void btnEntrar_Click(object sender, EventArgs e)
{
    // tudo que estiver aqui dentro roda quando o usuario clicar
}
```

O nome do método é montado pelo Visual Studio: nome do controle + underline + nome do evento.

LinkLabel — texto que vira link

Parece um Label, mas se comporta como um link de site: sublinhado, colorido e clicável.

Atenção: o evento não é Click

O LinkLabel usa o evento LinkClicked. É o erro mais comum com esse controle.

```
private void lnkCadastrar_LinkClicked(  
    object sender,  
    LinkLabelLinkClickedEventArgs e)  
{  
}
```

Propriedades de aparência

LinkColor cor normal do link

ActiveLinkColor cor no momento do clique

VisitedLinkColor cor depois de já ter sido clicado

LinkBehavior quando o sublinhado aparece

LinkVisited marca o link como já visitado

Panel — a caixa que organiza

O Panel é um controle que guarda outros controles dentro dele. É a base do nosso layout.

1

Move tudo junto

Arrastando o painel, todos os controles de dentro vão juntos, sem perder o alinhamento.

2

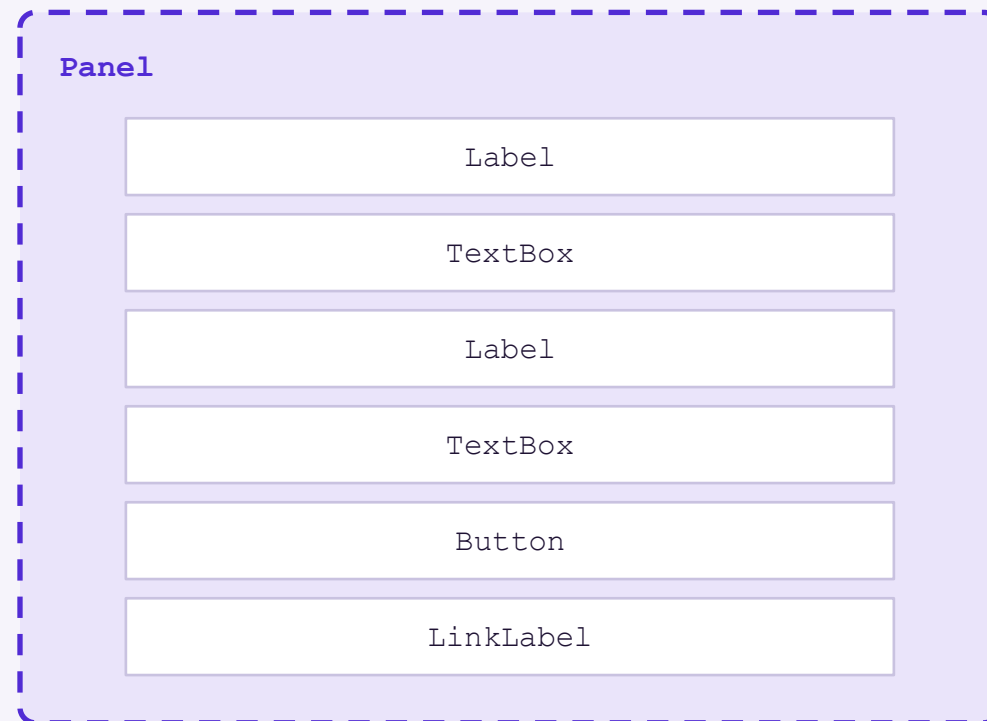
Esconde tudo junto

`Panel.Visible = false` some com o grupo inteiro de uma vez.

3

Facilita centralizar

Em vez de centralizar cinco controles, você centraliza apenas um.



Para colocar um controle dentro do painel, recorte (Ctrl+X) e cole (Ctrl+V) com o painel selecionado.

A propriedade Name

É o nome pelo qual você chama o controle no código. Sem um padrão, o projeto vira uma bagunça em poucas telas.

Sem padrão: `textBox1, textBox2, label3, button1...`

Com padrão: `txtEmail, txtSenha, btnEntrar...`

Controle	Prefixo	Exemplo
Label	<code>lbl</code>	<code>lblEmail</code>
TextBox	<code>txt</code>	<code>txtSenha</code>
Button	<code>btn</code>	<code>btnEntrar</code>
LinkLabel	<code>lnk</code>	<code>lnkCadastrar</code>
Panel	<code>pnl</code>	<code>pnlEntrar</code>
Form	<code>frm</code>	<code>frmCadastro</code>

1 Centralizando o painel

Queremos que o conteúdo fique no meio da tela mesmo quando o usuário redimensiona a janela.

Antes de programar

- Coloque todos os controles dentro do Panel
- No Panel, defina Anchor = None
- Deixe AutoSize = False

Por que Anchor = None?

O Anchor gruda o controle nas bordas do formulário. Com None, o painel fica livre e obedece só à posição que o código calcular.

```
private void CentralizarPainel()  
{  
    pnlEntrar.Left =  
        (this.ClientSize.Width  
        - pnlEntrar.Width) / 2;  
  
    pnlEntrar.Top =  
        (this.ClientSize.Height  
        - pnlEntrar.Height) / 2;  
}
```

```
this.Resize += (s, e) =>  
    CentralizarPainel();
```

Use ClientSize, não Width: o Width inclui a borda e a barra de título, e o cálculo sai torto.

2

Impedindo que a janela encolha demais

Sem limite, o usuário diminui a janela até os controles sumirem da tela.

O problema

O painel continua centralizado, mas o formulário fica menor que ele. O botão e o link ficam para fora da área visível.

A solução

A propriedade `MinimumSize` do formulário. O Windows simplesmente não deixa arrastar a borda além desse valor.

Detalhe importante

`MinimumSize` mede a janela inteira, com bordas e barra de título. Some uma folga generosa na altura, senão o conteúdo encosta no rodapé.

```
// forma simples
this.MinimumSize = new Size(
    pnlEntrar.Width + 40,
    pnlEntrar.Height + 100);
```

Onde colocar essa linha

```
private void Form1_Load(
    object sender, EventArgs e)
{
    DefinirTamanhoMinimo();
    CentralizarPainel();
}
```

3

Guardando um usuário de teste

Ainda não temos banco de dados. Vamos criar um email e uma senha fixos só para testar o login.

```
public partial class Form1 : Form
{
    private const string EmailCadastrado
        = "aluno@escola.com";

    private const string SenhaCadastrada
        = "1234";
```

Por que const?

const guarda um valor que nunca muda enquanto o programa roda. Se alguém tentar alterá-lo no código, o próprio Visual Studio acusa o erro antes de compilar.

Para os alunos, uma variável string comum também funciona.

Isto serve apenas para aprender

Em um sistema real, a senha fica no banco de dados e criptografada (hash). Senha escrita direto no código pode ser lida por qualquer pessoa que tenha acesso ao arquivo do programa.

4

Validando no clique do botão

Comparamos o que foi digitado com o usuário de teste e respondemos ao usuário.

Início do método — os campos estão preenchidos?

```
private void btnEntrar_Click(
    object sender, EventArgs e)
{
    string email =
        txtEmail.Text.Trim();
    string senha = txtSenha.Text;

    if (email == "" || senha == "")
    {
        MessageBox.Show(
            "Preencha o email e a senha.",
            "Atencao",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        return;
    }
}
```

Continuação — os dados conferem?

```
if (email.ToLower() == EmailCadastrado
    && senha == SenhaCadastrada)
{
    MessageBox.Show(
        "Login realizado com sucesso!",
        "Bem-vindo", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
}
else
{
    MessageBox.Show(
        "Email ou senha incorretos.",
        "Erro", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    txtSenha.Clear();
    txtSenha.Focus();
}
}
```

É um método só: a coluna da direita continua exatamente de onde a da esquerda parou.

Três detalhes que valem ouro

São pequenos, passam despercebidos e evitam horas de dor de cabeça.

`Trim()`

Remove os espaços em branco do começo e do fim.

O aluno digita o email com um espaço no final sem perceber, jura que está certo e o login não entra.

`ToLower()`

Deixa tudo minúsculo antes de comparar.

Email não diferencia maiúscula de minúscula. Senha diferencia — por isso não usamos ToLower nela.

`return`

Interrompe o método ali mesmo.

Sem ele, o código continuaria correndo e o usuário veria duas mensagens seguidas na tela.

MessageBox — falando com o usuário

A mesma linha de código muda completamente de cara conforme os botões e o ícone que você escolhe.

MessageBoxButtons

<code>OK</code>	OK
<code>OKCancel</code>	OK Cancelar
<code>YesNo</code>	Sim Não
<code>YesNoCancel</code>	Sim Não Cancelar
<code>RetryCancel</code>	Repetir Cancelar
<code>AbortRetryIgnore</code>	Anular Repetir Ignorar

MessageBoxIcon

-  **Information** Sucesso, aviso neutro
-  **Warning** Alerta, campo em branco
-  **Error** Erro, falha na operação
-  **Question** Pedido de confirmação
-  **None** Mensagem simples, sem ícone

```
MessageBox.Show(texto, titulo,
                botoes, icone);
```

DialogResult — a resposta do usuário

O MessageBox não só mostra a mensagem: ele devolve qual botão foi clicado.

```
DialogResult resposta = MessageBox.Show(  
    "Deseja realmente sair?",  
    "Confirmacao",  
    MessageBoxButtons.YesNo,  
    MessageBoxIcon.Question);  
  
if (resposta == DialogResult.Yes)  
{  
    Application.Exit();  
}
```

Valores possíveis

DialogResult.OK

DialogResult.Cancel

DialogResult.Yes

DialogResult.No

DialogResult.Retry

DialogResult.Abort

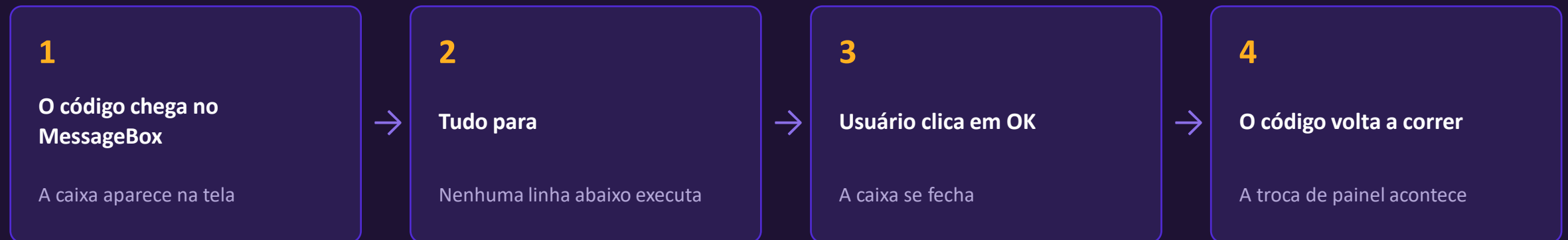
DialogResult.Ignore

Esse exemplo do "deseja sair?" costuma render em sala: o aluno já viu isso em dezenas de programas e nunca tinha parado para pensar em como se faz.

O detalhe que explica tudo

O MessageBox congela a execução do código.

A linha seguinte fica parada, esperando. Só quando o usuário clica em OK é que ela roda.



É exatamente o mesmo comportamento do `ShowDialog()` que veremos a seguir.

5

Abrindo o formulário de cadastro

O LinkLabel "Cadastrar-se" leva a uma tela nova, com nome, email e senha.

Criando o formulário

- Botão direito no projeto, no Solution Explorer
- Adicionar → Novo Item → Windows Form
- Nomeie como FrmCadastro.cs
- Monte os campos usando um Panel, como já fizemos

Voltando ao login

```
// no link "Ja tenho conta"  
this.Close();
```

```
private void lnkCadastrar_LinkClicked(  
    object sender,  
    LinkLabelLinkClickedEventArgs e)  
{  
    FrmCadastro cadastro =  
        new FrmCadastro();  
  
    this.Hide();  
    cadastro.ShowDialog();  
    this.Show();  
}
```

Não crie um novo Form1 para voltar. O login continua vivo, apenas escondido — basta mostrá-lo de novo.

Por que escrevemos o nome três vezes?

```
FrmCadastro cadastro = new FrmCadastro();
```

FrmCadastro

O tipo

Que espécie de coisa é essa. É o nome real da classe — esse você não escolhe.

cadastro

O nome

Como você vai chamá-la no seu código. Esse é livre: podia ser batata e funcionaria igual.

new FrmCadastro()

A criação

Constrói o objeto de verdade na memória. Sem o new, a tela simplesmente não existe.

Compare com o que eles já conhecem: `string nome = "João";` — mesma forma: tipo, nome, valor.

A planta e a casa

Uma forma simples de explicar objetos para quem nunca programou.

A classe é a planta

```
FrmCadastro
```

Você desenhou os botões e campos no designer. Mas planta não é casa — ninguém mora dentro de um papel.

new constrói a casa

```
new FrmCadastro()
```

É neste instante que a tela passa a existir de verdade na memória do computador.

A variável é o endereço

```
cadastro
```

É a anotação para você achar a casa depois. Sem ela, você constrói e não consegue mais chamar de volta.

Por isso dá para abrir a mesma tela duas vezes: duas casas, mesma planta, cada uma com seus próprios campos preenchidos.

Show() ou ShowDialog()?

As duas abrem uma janela. A diferença está no que acontece com o código depois.

Show()

Abre a janela e o código continua correndo.

As duas telas ficam utilizáveis ao mesmo tempo. O usuário pode ir e voltar entre elas livremente.

Bom para: janelas auxiliares, ajuda, consulta rápida.

ShowDialog()

Abre a janela e o código para ali.

A tela anterior fica bloqueada até a nova ser fechada. Só então a linha seguinte executa.

Bom para: login, cadastro, confirmações.

Para o cadastro usamos ShowDialog(): não queremos ninguém mexendo no login enquanto preenche o formulário.

6

Email e senha viram variáveis do cadastro

Antes eram fixos com const. Agora quem define os valores é o formulário de cadastro — por isso viram public static.

```
public partial class Form1 : Form
{
    public static string NomeCadastrado
        = "";

    public static string EmailCadastrado
        = "aluno@escola.com";

    public static string SenhaCadastrada
        = "1234";
```

Por que public static?

static faz a variável pertencer à classe inteira, não a uma tela específica. Por isso o FrmCadastro consegue escrever em Form1.EmailCadastrado sem precisar de uma instância do Form1 já aberta.

public libera esse acesso para outras classes — sem ele, só o próprio Form1 enxergaria a variável.

Enquanto ninguém se cadastra

O login continua funcionando com os valores iniciais (aluno@escola.com / 1234). Assim que alguém preencher o cadastro, esses valores são substituídos pelos novos dados digitados.

7 Código do botão Cadastrar

Valida os campos, salva os dados nas variáveis do Form1 e volta para a tela de login.

Início do método — os campos estão preenchidos?

```
private void btnCadastrar_Click(
    object sender, EventArgs e)
{
    string nome = txtNome.Text.Trim();
    string email = txtEmail.Text.Trim();
    string senha = txtSenha.Text;
    if (nome == "" || email == "" ||
        senha == "")
    {
        MessageBox.Show(
            "Preencha todos os campos.",
            "Atencao",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        return;
    }
}
```

Continuação — salvando os dados e voltando ao login

```
// salva os dados para o login
Form1.NomeCadastrado = nome;
Form1.EmailCadastrado =
    email.ToLower();
Form1.SenhaCadastrada = senha;

MessageBox.Show(
    "Cadastro realizado!",
    "Sucesso", MessageBoxButtons.OK,
    MessageBoxIcon.Information);

// Form1 so estava escondido,
// entao nao criamos um novo
this.Close();
}
```

É um método só: a coluna da direita continua exatamente de onde a da esquerda parou.

8 Preparando o FrmPrincipal

Depois do login, quem assume a tela é um formulário novo — o mesmo truque que já usamos no cadastro.

Criando o formulário

- Botão direito no projeto, no Solution Explorer
- Adicionar → Novo Item → Windows Form
- Nomeie como FrmPrincipal.cs
- Monte um Label lblBoasVindas e um Button btnSair

Por que um Form novo?

É o mesmo padrão do FrmCadastro: cada tela do sistema tem o seu próprio formulário. Assim o Form1 continua responsável só pelo login, e o FrmPrincipal cresce sozinho conforme o sistema for ganhando mais telas.

Não é um conceito novo: é Hide() + Show() de novo, só que desta vez saindo do login para o sistema em si.

9

Código: abrindo o FrmPrincipal

Login correto cria o FrmPrincipal, entrega o nome e esconde o Form1.

No Form1 — dentro do if de login correto

```
if (email.ToLower() ==  
    EmailCadastrado  
    && senha == SenhaCadastrada)  
{  
    FrmPrincipal principal =  
        new FrmPrincipal();  
  
    principal.DefinirBoasVindas(  
        NomeCadastrado);  
  
    principal.Show();  
    this.Hide();  
}
```

No FrmPrincipal — recebendo o nome

```
public partial class FrmPrincipal  
    : Form  
{  
    public FrmPrincipal()  
    {  
        InitializeComponent();  
    }  
  
    public void DefinirBoasVindas(  
        string nome)  
    {  
        lblBoasVindas.Text =  
            "Bem-vindo, " + nome + "!";  
    }  
}
```

Show() (não ShowDialog): o login fica escondido em segundo plano enquanto o FrmPrincipal assume a tela.

10

Cadastro: confirmar senha e voltar ao login

Dois ajustes pequenos no FrmCadastro, sem mexer no Form1.

Novo TextBox txtConfirmarSenha, dentro do btnCadastrar_Click

```
if (nome == "" || email == "" ||  
    senha == "" ||  
    txtConfirmarSenha.Text == "")  
{  
    MessageBox.Show(  
        "Preencha todos os campos.",  
        "Atencao",  
        MessageBoxButtons.OK,  
        MessageBoxIcon.Warning);  
    return;  
}  
  
if (senha != txtConfirmarSenha.Text)  
{  
    MessageBox.Show(  
        "As senhas nao conferem.",  
        "Atencao",  
        MessageBoxButtons.OK,  
        MessageBoxIcon.Warning);  
    return;  
}
```

Novo LinkLabel lnkVoltar, texto "Já tenho conta"

```
private void lnkVoltar_LinkClicked(  
    object sender,  
    LinkLabelLinkClickedEventArgs e)  
{  
    this.Close();  
}  
  
// o Form1 continua vivo, so  
// estava escondido (Hide) -  
// ao fechar o cadastro, ele  
// aparece de novo sozinho
```

As senhas comparam com != (diferente de), o resto é igual ao que já fizemos no login.

11

Saindo do sistema com confirmação

Um evento pergunta e decide; o outro fecha a aplicação de vez.

FormClosing — pergunta e decide se cancela

```
private void frmPrincipal_FormClosing(
    object sender,
    FormClosingEventArgs e)
{
    DialogResult resposta =
        MessageBox.Show(
            "Deseja realmente sair?",
            "Confirmacao",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

    if (resposta == DialogResult.No)
    {
        e.Cancel = true;
    }
}
```

FormClosed — fecha a aplicação de vez

```
private void frmPrincipal_FormClosed(
    object sender,
    FormClosedEventArgs e)
{
    Application.Exit();
}

// dispara so depois que a
// janela ja fechou de verdade,
// entao nao ha mais risco de
// reabrir o FormClosing
```

Application.Exit() dentro do FormClosing reabre o próprio evento — por isso pedia confirmação 2x. Separado no FormClosed, o problema some.